پروژه پایانی
پردازش زبان های طبیعی
۴ اسفندماه ۱۴۰۴

تحلیل احساسات متن‌های فارسی با استفاده از ترکیب روش‌های کلاسیک و مدل‌های زبانی زمینه‌ای (Contextual Embeddings)

امیرحسین وکیل زاده طبسی

گروه آموزشی کارشناسی ارشد کامپیوتر – هوش مصنوعی، دانشگاه غیر انتفاعی خیام، مشهد

Amirhosseinvakilzadeh82@gmail.com

چکیده

تحلیل احساسات (Sentiment Analysis) یکی از مهم‌ترین کاربردهای پردازش زبان طبیعی است که در حوزه‌های تجارت الکترونیک، نظرسنجی‌ها و نظارت بر رسانه‌های اجتماعی کاربرد گسترده‌ای دارد. در این پروژه، یک سیستم تحلیل احساسات سه‌کلاسه (منفی، مثبت، خنثی) برای زبان فارسی طراحی و پیاده‌سازی شد.

ابتدا داده‌های آموزشی و آزمایشی از مجموعه sentiment_dksf بارگذاری و پیش‌پردازش گردید. مراحل پیش‌پردازش شامل نرمال‌سازی، حذف نویز، توکنایز با کتابخانه Hazm و حذف stopwords سفارشی بود. سپس چهار روش نمایش متن شامل TF-IDF ، Word2Vec، FastText و ParsBERT پیاده‌سازی و مقایسه شدند. نتایج نشان داد ParsBERT پوشش ۱۰۰٪ و TF-IDF پوشش ۳۴/۵۲٪ از واژگان را دارند.

در بخش مدل‌سازی، ترکیب‌های مختلفی از جمله TF-IDF + Random Forest ، ParsBERT + Logistic Regression، خوشه‌بندی KMeans روی embeddingهای ParsBERT و بهینه‌سازی با GridSearchCV بررسی شد. بهترین عملکرد متعلق به مدل TF-IDF + Random Forest با GridSearchCV با Accuracy ۸۴/۳۹٪ و F1-macro ۸۰/۰۷٪ بود. مدل Baseline (TF-IDF + Naive Bayes) نیز برای مقایسه پیاده‌سازی شد.

نتایج نشان داد روش‌های کلاسیک (به‌ویژه Random Forest) در این دیتاست عملکرد بهتری نسبت به مدل زبانی بدون fine-tune دارند. در نهایت، نقاط قوت و ضعف هر رویکرد تحلیل و پیشنهادهایی برای کارهای آینده ارائه گردید.

کلمات کلیدی

تحلیل احساسات، پردازش زبان طبیعی فارسی، TF-IDF، ParsBERT، Random Forest

۱ ــ مقدمه

با گسترش پلتفرم‌های تجارت الکترونیک و شبکه‌های اجتماعی، حجم عظیمی از نظرات کاربران به صورت متنی تولید می‌شود. تحلیل خودکار این نظرات می‌تواند به شرکت‌ها کمک کند تا نظرات مشتریان را به سرعت درک کنند و تصمیم‌گیری‌های بهتری بگیرند.

هدف این پروژه، طراحی و پیاده‌سازی یک سیستم تحلیل احساسات سه‌کلاسه برای زبان فارسی است. دیتاست مورد استفاده شامل نظرات کاربران در حوزه‌های مختلف است که برچسب‌گذاری شده‌اند (۰=منفی، ۱=مثبت، ۲=خنثی).

اهمیت این کار در دو بعد است:

- جنبه کاربردی: کمک به کسب‌وکارها برای تحلیل نظرات مشتریان
- جنبه علمی: مقایسه عملکرد روش‌های کلاسیک و مدل‌های زبانی عمیق در زبان فارسی که منابع کمتری نسبت به انگلیسی دارد.

۲ ـ مرور کوتاه ادبیات

تحلیل احساسات یکی از قدیمی‌ترین وظایف NLP است. روش‌های اولیه بر پایه Bag-of-Words و TF-IDF بودند. با ظهور مدل‌های زبانی، روش‌هایی مانند Word2Vec، GloVe و سپس مدل‌های Transformer مانند BERT تحول بزرگی ایجاد کردند.

در زبان فارسی، مدل ParsBERT (HooshvareLab, 2020) یکی از قوی‌ترین مدل‌های پیش‌آموزش‌دیده است. مطالعات متعدد نشان داده‌اند که در دیتاست‌های کوچک یا نویزدار، ترکیب TF-IDF با طبقه‌بندهای کلاسیک (مانند Random Forest) گاهی عملکرد بهتری نسبت به BERT بدون fine-tune دارند.

۳ ـ دادگان

دیتاست مورد استفاده شامل دو فایل Parquet است:

- آموزشی: ۲۸۶۰۲ نمونه (پس از پیش‌پردازش: ۲۸۴۴۱ نمونه)
- آزمایشی: ۲۳۱۵ نمونه (پس از پیش‌پردازش: ۲۳۱۲ نمونه)

توزیع کلاس‌ها در داده آموزشی تقریباً متعادل است.

- ۰: ۱۰۲۰۶
- ۱: ۹۵۸۷
- ۲: ۸۸۰۹

- منبع دیتاست: [https://huggingface.co/datasets/hezarai/sentiment-dksf]

- لینک پروژه در گیت هاب: [https://github.com/amir-v-z/Project-NLP]

آمار اولیه:

- میانگین طول متن خام: ۱۲۵/۵۳ کاراکتر
- میانگین طول متن پس از پیش‌پردازش: ۹۸/۲۱ کاراکتر

۴ ـ روش انجام کار

۴-۱ پیش‌پردازش متن روی داده های آموزشی

- نرمال‌سازی با کتابخانه Hazm
- حذف URL ، علائم نگارشی و اعداد
- توکنایز و حذف stopwords سفارشی (حروف اضافه، افعال کمکی و کلمات عمومی)
- اتصال توکن‌ها به یک رشته واحد (clean_text_joined)
- حذف داده های NaN و رشته های خالی (مجموعا ۱۶۱ ردیف حذف شد)

۴-۲ نمایش متن داده های آموزشی (Embedding)

چهار روش پیاده‌سازی شد:

1. **TF-IDF:** max_features=۱۵۰۰۰

   (پوشش ۳۴/۵۲٪)

2. **FastText:** vector_size=۳۰۰

   (پوشش بسیار پایین ≈ ۰/۲۵٪)

3. **Word2Vec:** vector_size=۳۰۰

   (پوشش بسیار پایین ≈ ۰/۲۴٪)

4. **ParsBERT:** mean pooling روی آخرین لایه

   (پوشش ۱۰۰٪)

مقایسه روش ها:

روش	ابعاد بردار	% پوشش	similarity
ParsBERT	۷۶۸	۱۰۰	۰/۶۳۱۶۴۶
TF-IDF	۱۵۰۰۰	۳۴/۰۵	۰/۰۰۰۰۰۰
FastText	۳۰۰	۰/۲۵	۰/۹۸۷۲۴۳
Word2Vec	۳۰۰	۰/۲۴	۰/۹۹۳۸۵۶

۴ـ۳ پیش پردازش متن روی داده های تستی

تمامی عملیات مرحله (۴ـ۱) انجام شده.

۴ـ۴ نمایش متن داده های تستی (Embedding)

دو روش پیاده‌سازی شد:

1. **TF-IDF**
2. **ParsBERT**

مقایسه روش ها:

نتایج TF-IDF روی داده های تستی:

	precision	recall	f1-score	support
۰	۰/۸۵	۰/۸۳	۰/۸۴	۱۱۰۷
۱	۰/۹۲	۰/۷۵	۰/۸۲	۱۰۳۲
۲	۰/۳۵	۰/۷۷	۰/۴۸	۱۷۳
Accuracy			۰/۷۹	۲۳۱۲
Marco avg	۰/۷۱	۰/۷۸	۰/۷۲	۲۳۱۲
Weighted avg	۰/۸۴	۰/۷۹	۰/۸۱	۲۳۱۲

نتایج ParsBERT روی داده های تستی:

	precision	recall	f1-score	support
۰	۰/۸۰	۰/۷۸	۰/۷۹	۱۱۰۷
۱	۰/۸۷	۰/۷۲	۰/۷۹	۱۰۳۲
۲	۰/۲۸	۰/۶۱	۰/۳۹	۱۷۳
Accuracy			۰/۷۴	۲۳۱۲
Marco avg	۰/۶۵	۰/۷۰	۰/۶۵	۲۳۱۲
Weighted avg	۰/۷۹	۰/۷۴	۰/۷۶	۲۳۱۲

۴-۵ مدل‌سازی

ترکیب‌های پیاده‌شده:

- TF-IDF + Random Forest
- TF-IDF + Random Forest + GridSearchCV
- ParsBERT + Logistic Regression
- ParsBERT + KMeans Clustering + Logistic Regression

مقایسه مدل های طبقه بندی روی داده های تستی:

روش	Accuracy	f1-marco
TF-IDF + RF + GridSearchCV	۸۴/۳۹	۸۰/۰۷
TF-IDF + RF	۸۴/۳۴	۷۹/۸۴
ParsBERT + KMeans + LR	۷۳/۹۶	۶۵/۱۰
ParsBERT + LR	۷۳/۳۱	۶۴/۷۳

۴-۶ Fine-tuning

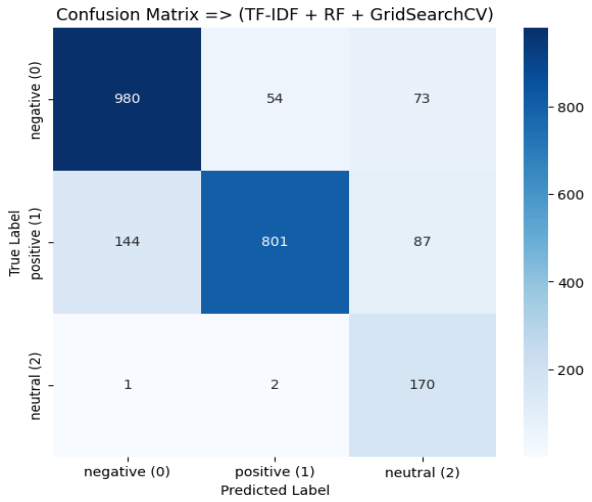
تلاش برای fine-tune مدل ParsBERT انجام شد، اما به دلیل محدودیت‌های سخت‌افزاری (مشکل کتابخانه accelerate) به صورت کامل اجرا نشد. (عکس خروجی در انتها قرار داده شده).

۵ـ نتایج و بحث

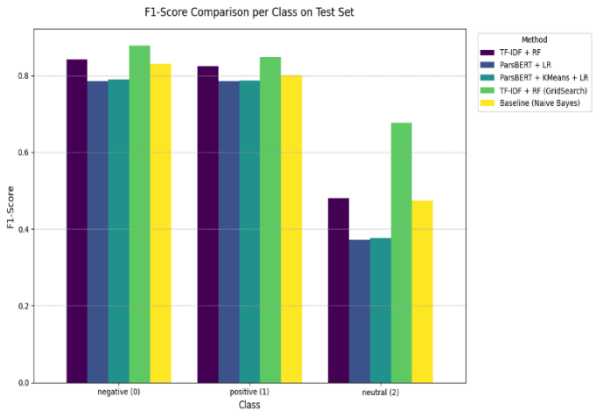
۵ ـ ۱ نمودار های ارزیابی

بهترین مدل: TF-IDF + Random Forest با GridSearchCV

- **Accuracy:** ٪۸۴/۳۹
- **F1-macro:** ٪۸۰/۰۷



شکل (۱): نمودار Confusion Matrix



شکل (۲): نمودار Bar Chart

۵ـ۲ مقایسه با baseline

مقایسه نهایی:

روش	Accuracy (%)	F1-macro (%)
TF-IDF + RF + GridSearchCV	۸٤/۹۳	۸۰/۰۷
TF-IDF + RF	۸۴/۳۴	۷۹/۸۴
Baseline: TF-IDF + Naive Bayes	۷۷/۸۵	۷۰/۱۸
ParsBERT + KMeans + LR	۷۳/۹۶	۶۵/۱۰
ParsBERT + LR	۷۳/۳۱	۶۴/۷۳

۵ـ۳ تحلیل

- مدل‌های مبتنی بر TF-IDF عملکرد بهتری داشتند که نشان‌دهنده مناسب بودن ویژگی‌های آماری برای این دیتاست خاص است.
- ParsBERT بدون fine-tune ضعیف‌تر عمل کرد که انتظار می‌رفت.
- کلاس neutral بیشترین خطا را داشت (مشاهده‌شده در Confusion Matrix).

۶ـ نتیجه‌گیری

در این پروژه یک سیستم کامل تحلیل احساسات برای زبان فارسی طراحی شد. نتایج نشان داد که برای دیتاست‌های متوسط، ترکیب TF-IDFبا Random Forest همچنان بسیار رقابتی است. مدل‌های زبانی مانند ParsBERT پتانسیل بالایی دارند ولی نیاز به fine-tuning روی دیتاست هدف دارند.

سپاسگزاری

از استاد محترم راهنما و مشاور پروژه، جناب خانم دکتر شریف، به دلیل راهنمایی‌های ارزشمند، حمایت علمی و صبر و حوصله بی‌پایان در طول انجام این پروژه صمیمانه سپاسگزارم. همچنین از زحمات و حمایت‌های دانشگاه غیرانتفاعی خیام مشهد و گروه آموزشی کامپیوتر تشکر می‌کنم.

مراجع

[1] Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers.

[2] Vaswani, A. et al. (2017). Attention is All You Need.

[3] Farhadi, et al. (2020). ParsBERT: Transformer-based Model for Persian Language Understanding.

[4] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python.

[5] Mikolov, T. et al. (2013). Efficient Estimation of Word Representations in Vector Space.

عکس بخش (۴ـ۶) Fine-tuning

```
trainer = Trainer(
    model=model_cls,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics,
)

print("...fine-tuning شروع")
trainer.train()
```

```
...fine-tuning شروع
c:\Users\amir_v_z\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\utils\data\dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accel
  super().__init__(loader)
```

[178/5334 22:41 < 11:04:51, 0.13 it/s, Epoch 0.10/3]

Epoch	Training Loss	Validation Loss

```
    )
    attention_output = cross_attention_output
--> 423 layer_output = apply_chunking_to_forward(
    self.feed_forward_chunk, self.chunk_size_feed_forward, self.seq_len_dim, attention_output
)
return layer_output

File c:\Users\amir_v_z\AppData\Local\Programs\Python\Python312\Lib\site-packages\transformers\pytorch_utils.py:201, in apply_chunking_to_forward(forward_fn, chunk_size, chu
    # concatenate output at same dimension
    return torch.cat(output_chunks, dim=chunk_dim)
--> 201 return forward_fn(*input_tensors)

File c:\Users\amir_v_z\AppData\Local\Programs\Python\Python312\Lib\site-packages\transformers\models\bert\modeling_bert.py:430, in BertLayer.feed_forward_chunk(self, attent
def feed_forward_chunk(self, attention_output):
    intermediate_output = self.intermediate(attention_output)
```